
Reflexion: Language Agents with Verbal Reinforcement Learning

Noah Shinn

Northeastern University
noahshinn024@gmail.com

Federico Cassano

Northeastern University
cassano.f@northeastern.edu

Edward Berman

Northeastern University
berman.ed@northeastern.edu

Ashwin Gopinath

Massachusetts Institute of Technology
agopi@mit.edu

Karthik Narasimhan

Princeton University
karthikn@princeton.edu

Shunyu Yao

Princeton University
shunyuy@princeton.edu

Abstract

Large language models (LLMs) have been increasingly used to interact with external environments (e.g., games, compilers, APIs) as goal-driven agents. However, it remains challenging for these language agents to quickly and efficiently learn from trial-and-error as traditional reinforcement learning methods require extensive training samples and expensive model fine-tuning. We propose *Reflexion*, a novel framework to reinforce language agents not by updating weights, but instead through linguistic feedback. Concretely, Reflexion agents verbally reflect on task feedback signals, then maintain their own reflective text in an episodic memory buffer to induce better decision-making in subsequent trials. Reflexion is flexible enough to incorporate various types (scalar values or free-form language) and sources (external or internally simulated) of feedback signals, and obtains significant improvements over a baseline agent across diverse tasks (sequential decision-making, coding, language reasoning). For example, Reflexion achieves a 91% pass@1 accuracy on the HumanEval coding benchmark, surpassing the previous state-of-the-art GPT-4 that achieves 80%. We also conduct ablation and analysis studies using different feedback signals, feedback incorporation methods, and agent types, and provide insights into how they affect performance. We release all code, demos, and datasets at <https://github.com/noahshinn024/reflexion>.

1 Introduction

Recent works such as ReAct [30], SayCan [1], Toolformer [22], HuggingGPT [23], generative agents [19], and WebGPT [17] have demonstrated the feasibility of autonomous decision-making agents that are built on top of a large language model (LLM) core. These methods use LLMs to generate text and ‘actions’ that can be used in API calls and executed in an environment. Since they rely on massive models with an enormous number of parameters, such approaches have been so far limited to using in-context examples as a way of teaching the agents, since more traditional optimization schemes like reinforcement learning with gradient descent require substantial amounts of compute and time.

In this paper, we propose an alternative approach called *Reflexion* that uses verbal reinforcement to help agents learn from prior failings. Reflexion converts binary or scalar feedback from the environment into verbal feedback in the form of a textual summary, which is then added as additional context for the LLM agent in the next episode. This self-reflective feedback acts as a ‘semantic’ gradient signal by providing the agent with a concrete direction to improve upon, helping it learn from prior mistakes to perform better on the task. This is akin to how humans iteratively learn to accomplish complex tasks in a few-shot manner – by reflecting on their previous failures in order to form an improved plan of attack for the next attempt. For example, in figure 1, a Reflexion agent learns to optimize its own behavior to solve decision-making, programming, and reasoning tasks through trial, error, and self-reflection.

Generating useful reflective feedback is challenging since it requires a good understanding of where the model made mistakes (i.e. the credit assignment problem [25]) as well as the ability to generate a summary containing actionable insights for improvement. We explore three ways for doing this – simple binary environment feedback, pre-defined heuristics for common failure cases, and self-evaluation such as binary classification using LLMs (decision-making) or self-written unit tests (programming). In all implementations, the evaluation signal is amplified to natural language experience summaries which can be stored in long-term memory.

Reflexion has several advantages compared to more traditional RL approaches like policy or value-based learning: 1) it is lightweight and doesn’t require finetuning the LLM, 2) it allows for more nuanced forms of feedback (e.g. targeted changes in actions), compared to scalar or vector rewards that are challenging to perform accurate credit assignment with, 3) it allows for a more explicit and interpretable form of episodic memory over prior experiences, and 4) it provides more explicit hints for actions in future episodes. At the same time, it does have the disadvantages of relying on the power of the LLM’s self-evaluation capabilities (or heuristics) and not having a formal guarantee for success. However, as LLM capabilities improve, we only expect this paradigm to get better over time.

We perform experiments on (1) decision-making tasks to test sequential action choices over long trajectories, (2) reasoning tasks to test knowledge-intensive, single-step generation improvement, and (3) programming tasks to teach the agent to effectively use external tools such as compilers and interpreters. Across all three types of tasks, we observe Reflexion agents are better decision-makers, reasoners, and programmers. More concretely, Reflexion agents improve on decision-making AlfWorld [24] tasks over strong baseline approaches by an absolute 22% in 12 iterative learning steps, and on reasoning questions in HotPotQA [28] by 20%, and Python programming tasks on HumanEval [6] by as much as 11%.

To summarize, our contributions are the following:

- We propose Reflexion, a new paradigm for ‘verbal’ reinforcement that parameterizes a policy as an agent’s memory encoding paired with a choice of LLM parameters.
- We explore this emergent property of *self-reflection* in LLMs and empirically show that self-reflection is extremely useful to learn complex tasks over a handful of trials.
- We introduce LeetCodeHardGym, a code-generation RL gym environment consisting of 40 challenging LeetCode questions (‘hard-level’) in 19 programming languages.
- We show that Reflexion achieves improvements over strong baselines across several tasks, and achieves state-of-the-art results on various code generation benchmarks.

2 Related work

Reasoning and decision-making Self-Refine [15] employs an iterative framework for self-refinement to autonomously improve generation through self-evaluation. These self-evaluation and self-improvement steps are conditioned on given task constraints, such as "How can this generation be written in a more positive way". Self-Refine is effective but is limited to single-generation reasoning tasks. Pryzant et al. [21] performs a similar semantic prompt-writing optimization, but is also limited to single-generation tasks. Paul et al. [20] fine-tune critic models to provide intermediate feedback within trajectories to improve reasoning responses. Xie et al. [27] use stochastic beam search over actions to perform a more efficient decision-making search strategy which allows the agent to use foresight advantage due to its self-evaluation component. Yoran et al. [31] and Nair et al.

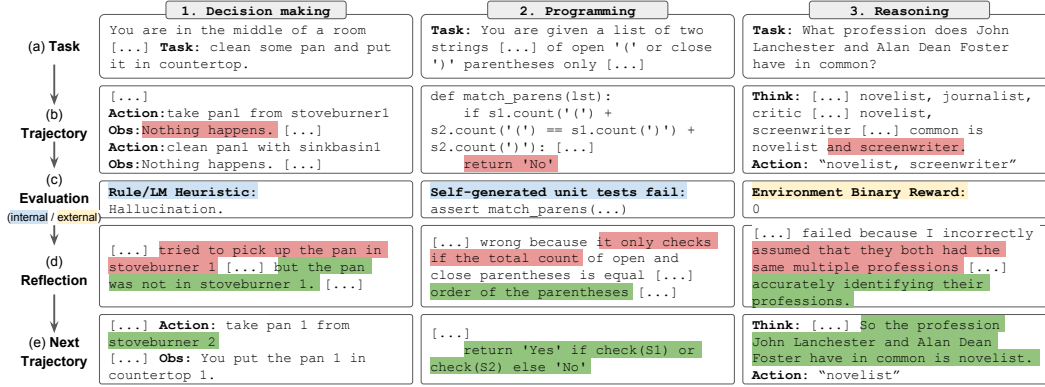


Figure 1: Reflexion works on decision-making 4.1, programming 4.3, and reasoning 4.2 tasks.

Related work on reasoning and decision-making					
Approach	Self refine	Hidden constraints	Decision making	Binary reward	Memory
Self-refine [15]	✓	✗	✗	✗	✗
Beam search [27]	✓	✓	✓	✓	✗
Reflexion (ours)	✓	✓	✓	✓	✓

Related work on programming					
Approach	Test execution	Debugging	Self-generated tests	Multiple languages	Self-reflection
AlphaCode [14]	✓	✗	✗	✓	✗
CodeT [5]	✓	✗	✓	✗	✗
Self-debugging [7]	✓	✓	✗	✗	✗
CodeRL [12]	✓	✓	✗	✗	✗
Reflexion (ours)	✓	✓	✓	✓	✓

[16] use decider models to reason over several generations. Kim et al. [10] use a retry pattern over a fixed number of steps without an evaluation step. Goodman [9] perform a qualitative evaluation step that proposes optimizations to the previous generation. In this paper, we show that several of these concepts can be enhanced with *self-reflection* to build a persisting memory of self-reflective experiences which allows an agent to identify its own errors and self-suggest lessons to learn from its mistakes over time.

Programming Several past and recent works employ variations of test-driven development or code debugging practices. AlphaCode [14] evaluates a set of generations on hidden test cases. CodeT [5] uses self-generated unit tests that are used to score generated function implementations. Self-Debugging [7] employs a debugging component that is used to improve existing implementations given feedback from a code execution environment. CodeRL [12] sets the problem in an RL framework using an actor-critic setup to debug programs given feedback from an execution environment. AlphaCode, Self-Debugging and CodeRL are effective in fixing less-complex program bugs, but they rely upon ground truth test cases that invalidate pass@1 eligibility, and do not use self-reflection to bridge the gap between error identification and implementation improvement. CodeT does not access hidden test cases but does not implement a self-learning step to improve code writing.

3 Reflexion: reinforcement via verbal reflection

We develop a modular formulation for Reflexion, utilizing three distinct models: an *Actor*, denoted as M_a , which generates text and actions; an *Evaluator* model, represented by M_e , that scores the outputs produced by M_a ; and a *Self-Reflection* model, denoted as M_{sr} , which generates verbal reinforcement cues to assist the Actor in self-improvement. We provide a detailed description of each of these models and subsequently elucidate their collaborative functioning within the Reflexion framework.